

ResNet (2015)

Tại sao mạng càng sâu thì độ chính xác lại càng giảm?
Trong các mạng thần kinh truyền thống (như VGG), dữ liệu phải đi qua hàng chục, hàng trăm lớp layer. Nhưng khi mạng quá sâu, thông tin từ những lớp đầu tiên dễ bị "rơi rụng" hoặc biến dạng khi đến được các lớp cuối.
Việc huấn luyện trở nên cực kỳ khó khăn, độ chính xác không những không tăng mà còn giảm đi (đây gọi là hiện tượng Degradation). Giống như việc bạn kể một câu chuyện qua 100 người, người thứ 100 chắc chắn sẽ nghe ra một nội dung hoàn toàn khác.

1. Giải pháp: "Đường tắt" (Skip Connection)

ResNet giải quyết vấn đề này bằng cách: Cung cấp một đường đi tắt. Thay vì bắt thông tin phải đi qua toàn bộ các lớp xử lý phức tạp, ResNet cho phép dữ liệu "nhảy cóc" qua một vài lớp và cộng trực tiếp vào kết quả phía sau.
Thay vì cố gắng học trực tiếp một hàm mục tiêu $H(x)$, ResNet thay đổi cách tiếp cận bằng cách học một hàm dư thừa (residual function)

$$H(x) = F(x) + x$$

Khi nhìn vào sơ đồ của một khối ResNet, bạn sẽ thấy:
- Đầu vào là x đi qua các lớp Convolution, ReLU,... để tạo ra $F(x)$ (phần dư).
- Đầu vào x được cộng trực tiếp vào kết quả đó.
- Kết quả cuối cùng thu được là $H(x)$



Các tác giả của ResNet đặt giả thuyết rằng: Nếu ta gọi hàm mục tiêu lý tưởng cần học là $H(x)$, thì thay vì bắt các lớp nơ-ron học trực tiếp $H(x)$, ta hãy bắt chúng học phần chênh lệch (dư thừa) giữa mục tiêu và đầu vào.
Phần chênh lệch đó chính là:

$$F(x) = H(x) - x$$

Tại sao lại gọi là "Skip" (Bỏ qua)?
- Mạng truyền thống: $A \rightarrow B \rightarrow C \rightarrow D$
- ResNet (Skip Layer): $A \rightarrow D$ (đi thẳng luôn) và $A \rightarrow B \rightarrow C \rightarrow D$ (đi qua đường xử lý khác)
Tại đích đến (D), người ta lấy kết quả của cả hai đường này cộng lại với nhau.

Hãy nhìn vào một "khối" (Residual Block) trong ResNet, dữ liệu sẽ bị chia làm hai phe tại ngã ba:
- Phe 1 (Đường chính): Vẫn phải đi qua đầy đủ các lớp Conv, ReLU, Batch Normalization... để cố gắng trích xuất các đặc trưng phức tạp (như hình dạng, mắt, mũi, miệng).
- Phe 2 (Đường tắt - Skip Connection): Đi thông dong trên con đường trống trải, không bị nhào nặn hay thay đổi gì cả.
Tại điểm có dấu (+), kết quả của hai phe này được cộng lại.
- Nếu các lớp Conv ở "Đường chính" học tốt: Nó sẽ đóng góp những thông tin mới, hữu ích vào tổng chung.
- Nếu các lớp Conv ở "Đường chính" học dốt (làm nhiễu dữ liệu): Trong quá trình huấn luyện, mạng sẽ tự động điều chỉnh để các lớp này có trọng số cực nhỏ (gần như bằng 0). Khi đó, kết quả tại điểm cộng chủ yếu là từ "Đường tắt" đưa sang.
-> Điều này làm cho mạng không còn bị ép buộc phải phụ thuộc hoàn toàn vào các lớp Conv đó nữa, trong các mạng cũ (như VGG), nếu một lớp Conv ở giữa bị thiết kế sai hoặc học tệ, toàn bộ các lớp phía sau sẽ nhận được "rác" và hỏng cả mô hình. Với ResNet, nếu lớp Conv đó học tệ, dữ liệu "sạch" từ đường tắt sẽ bắc cầu đi qua đoạn đó để nuôi sống các lớp phía sau.

2. Thiết kế kích thước ma trận để thực hiện phép cộng

Giả sử ta có 1 ảnh đầu vào với kích thước 10x10. Khi dữ liệu đi qua đường chính (Conv, ReLu, MaxPooling) thì kích thước hình ảnh ban đầu sẽ giảm đi 1 nửa đúng không, và khi truyền đi đường tắt thì kích thước ảnh vẫn giữ nguyên đúng không, vậy thì ta sẽ cộng ma trận của 2 đường dữ liệu này như thế nào khi kích thước khác nhau?
- Kích bản "Cùng kích thước" (Identity Block): Nếu các lớp Conv ở đường chính được thiết kế để giữ nguyên kích thước (ví dụ dùng padding='same'), thì ảnh 10x10 đi qua Conv vẫn ra 10x10
- Kích bản "Khác kích thước" (Convolutional Block): Nếu đường chính có Stride hoặc Pooling làm ảnh giảm xuống còn 5x5, trong khi đường tắt vẫn mang bản sao 10x10 tới, thì chúng ta không thể cộng được.
Để giải quyết, ResNet sẽ "phẫu thuật" đường tắt một chút:
- Họ không để đường tắt đi thẳng hoàn toàn nữa. Họ thêm vào đường tắt một lớp Convolution với kích thước kernel 1x1, stride = 2 (gọi là 1x1 Conv).Lớp 1x1 Conv này có nhiệm vụ: Thu nhỏ kích thước của đường tắt từ 10x10 xuống đúng 5x5 để vừa khít với đường chính.

3. Batch Normalization

Trong sơ đồ ResNet, BN thường nằm ngay sau lớp Convolution và trước ReLU:

Conv -> BN -> ReLU -> Conv -> BN -> (Cộng với Skip Connection) -> ReLU

Trong ResNet, Batch Normalization (BN) giống như một "người kiểm soát", đảm bảo các con số trong ma trận không bị quá lớn hoặc quá nhỏ khi đi qua hàng trăm lớp. Nếu không có sự kiểm soát, đến công đoạn thứ 50 các con số bắt đầu quá lớn (1000) hoặc quá nhỏ (0.000001) mạng sẽ ngừng học.
Để đơn giản nhất, hãy tưởng tượng chúng ta có một Batch gồm 3 tấm ảnh. Sau khi đi qua lớp Conv, mỗi tấm ảnh chỉ trích xuất ra đúng 1 con số duy nhất (để chúng ta dễ tính toán)
Ảnh 1: giá trị là 10
Ảnh 2: giá trị là 50
Ảnh 3: giá trị là 90
Trong quá trình tính toán BN chúng ta sẽ thực hiện theo 4 bước mà máy tính vẫn làm:
1. Tính trung bình:
Trung bình của cả mẻ (Batch) này là: $\mu = \frac{10 + 50 + 90}{3} = \frac{150}{3} = 50$
2. Tính phương sai:
Phương sai đo mức độ "phân tán" của các con số so với số 50
- Bình phương khoảng cách từ mỗi số đến số 50:

- Ảnh 1: $(10 - 50)^2 = (-40)^2 = 1600$
- Ảnh 2: $(50 - 50)^2 = 0^2 = 0$
- Ảnh 3: $(90 - 50)^2 = 40^2 = 1600$

- Trung bình của các bình phương này:
 $\sigma^2 = \frac{1600 + 0 + 1600}{3} = \frac{3200}{3} \approx 1066.67$
3. Tính Độ lệch chuẩn:
 $\sigma = \sqrt{1066.67} \approx 32.66$
4. Chuẩn hóa (Z-score):
Công thức là: $\hat{x} = \frac{x-\mu}{\sigma}$ (Lấy giá trị trừ trung bình rồi chia cho độ lệch chuẩn).

- Ảnh 1: $\hat{x}_1 = \frac{10-50}{32.66} = \frac{-40}{32.66} \approx -1.22$
- Ảnh 2: $\hat{x}_2 = \frac{50-50}{32.66} = \frac{0}{32.66} = 0$
- Ảnh 3: $\hat{x}_3 = \frac{90-50}{32.66} = \frac{40}{32.66} \approx 1.22$

Kết quả: Bạn thấy đấy, từ những con số "nhảy múa" tện 10 và 90, sau khi qua BN, chúng đều được đưa về quanh mức 0, với khoảng cách cực kỳ hẹp (từ -1.22 đến 1.22).

Bước cuối cùng: Scale (γ) và Shift (β)

Đây là điểm thông minh của BN. Sau khi đưa về -1.22, 0, 1.22, mạng sẽ tự hỏi: "Liệu đưa về trung bình = 0 có phải là tốt nhất cho việc học không?"

Mạng sẽ học thêm 2 tham số γ (nhân vào) và β (cộng vào):

$$Output = \gamma \cdot \hat{x} + \beta$$

- Nếu mạng thấy dễ nguyên là tốt, nó sẽ học $\gamma = 1$ và $\beta = 0$.
- Nếu mạng thấy cần số lớn hơn, nó sẽ tự điều chỉnh γ và β .

Tại sao làm vậy lại giúp ResNet học tốt?
Hãy nhìn vào giá trị của Ảnh 1 sau chuẩn hóa là -1.22. Nếu không có BN, giá trị là 10. Khi đi qua các lớp tiếp theo, số 10 này có thể bị nhân lên thành 100, 1000... (bùng nổ). Với BN, nó luôn bị "kéo" về vùng xung quanh số 0. Trong toán học, khi các con số ổn định xung quanh số 0, việc tính đạo hàm để cập nhật định trọng số sẽ cực kỳ mượt mà, không bị rơi vào tình trạng đạo hàm quá lớn làm hỏng mạng (Exploding Gradient) hoặc quá nhỏ làm mạng đứng im (Vanishing Gradient).
Việc đưa các con số về "chuẩn" như thế này giúp các phép cộng (+) trong ResNet trở nên "dễ thở" hơn. Nếu một bên là 1.000.000 (đường chính) mà cộng với 0.1 (đường tắt) thì con số 0.1 coi như vô nghĩa đúng không.

Tại sao ResNet-50 lại có ít tham số hơn nhiều so với VGG-16?
Có một sự thật thú vị: VGG-16 có tới khoảng 138 triệu tham số, dù nó chỉ có 16 lớp. Trong khi đó, ResNet-50 sâu hơn nhiều nhưng chỉ có khoảng 25.6 triệu tham số.
Lý do là:
- Loại bỏ các lớp Fully Connected (FC) khổng lồ: VGG sử dụng các lớp FC ở cuối với hàng ngàn neuron, chiếm phần lớn tham số. ResNet sử dụng Global Average Pooling để thay thế, giúp giảm cực mạnh số lượng tham số ở đầu ra.
- Cấu trúc Bottleneck: Ở các bản ResNet-50 trở lên, lớp Convolution 1x1 giúp nén số lượng kênh lại trước khi đưa qua lớp 3x3, giúp giữ số lượng tham số ở mức thấp dù mạng rất sâu.

Phiên bản	Số lượng tham số (xấp xỉ)	Đặc điểm kiến trúc
ResNet-18	11.7 triệu	Sử dụng Basic Block (2 lớp Conv 3 × 3)
ResNet-34	21.8 triệu	Sử dụng Basic Block, nhiều khối hơn ResNet-18
ResNet-50	25.6 triệu	Chuyển sang Bottleneck Block (1 × 1, 3 × 3, 1 × 1)
ResNet-101	44.5 triệu	Tăng số lượng Bottleneck Block ở giai đoạn 3
ResNet-152	60.2 triệu	Phiên bản sâu nhất trong các bản gốc